

UCS503 - Software Engineering

Lumen: An AI-Native MLOps System

Dataset Intelligence and Experiment Tracking - A Prototype Progress Report

Submitted by:

1024170404 Maitreya Gupta

1024170285 Varun Gupta

1024170266 Krrish Sood

1024170268 Gitansh

Group: XXXX BE Third Year, CSE

Submitted to:

Dr. Faculty Member Name

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	4
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	5
2.1.2	Technical Stack	6
2.2	Implementation Details	6
2.2.1	Module 1: Core Functionality	6

Chapter 1

Introduction

1.1 Background and Context

Machine learning teams increasingly rely on experiment-tracking tools such as MLflow, Weights & Biases, Neptune.ai, and Comet ML to log parameters, metrics, and artifacts generated while training models. These tools have made experiment record-keeping far more disciplined than it was a decade ago, but the underlying workflow is still largely manual: an engineer uploads a dataset, launches a training run, and then has to personally inspect logs, plots, and metric tables to work out whether the run succeeded, why it behaved the way it did, and what to try next. As the number of experiments and the size of the team grow, this dashboard-scrolling style of working does not scale, and hard-won lessons from past runs are rarely captured in a form that the next engineer -- or the next project -- can reuse.

Lumen is proposed as an AI-native MLOps platform that keeps the parts of MLflow-class tools that already work well -- structured tracking of datasets, runs, parameters, and metrics -- while adding a layer of active intelligence on top: automatic dataset profiling before training ever starts, and, in later phases of the project, automatic root-cause analysis, explainability, and an AI copilot. This report documents the design and the first working slice of that system, built as a capstone project for the Software Engineering (UCS503) course at Thapar Institute of Engineering and Technology.

1.1.1 Problem Statement

Teams that train machine learning models repeatedly hit four related problems that current tracking tools do not solve on their own:

- **Passive tracking:** existing tools store parameters and metrics but do not interpret them -- there is no automatic explanation of why a run performed the way it did.
- **No institutional memory:** insights from earlier experiments are rarely captured in a reusable form, so the same mistakes (bad hyperparameters, leaky features) are repeated across engineers and projects.
- **Late-discovered data issues:** problems such as missing values, duplicate rows, class imbalance, or outliers are usually discovered only after a training run has already been wasted on bad data.
- **Fragmented workflow:** uploading a dataset, understanding its quality, configuring a model, and reading the resulting metrics typically happen in disconnected tools and spreadsheets rather than one coherent pipeline.

Lumen's problem statement, scoped for this stage of the project, is therefore: *make the dataset-to-metric pipeline of an ML experiment fully connected and self-explanatory*, starting with the two modules that sit at the foundation of every later intelligence feature -- Dataset Intelligence and Experiment Tracking with real model training.

1.2 Project Objectives

- 1 **Objective 1 (Dataset Intelligence):** Allow a user to upload a real CSV dataset and have the backend profile it automatically -- row/column counts, column types, missing values, duplicate rows, descriptive statistics, and basic data-quality flags -- and persist that profile so it can be reused by later modules, within the current development sprint.
- 2 **Objective 2 (Experiment Tracking and Training):** Allow a user to select an uploaded dataset, choose a target column, a model type, and hyperparameters, and have the backend actually train a real scikit-learn model on that dataset, returning genuine evaluation metrics (accuracy, precision, recall, F1) rather than simulated ones.
- 3 **Objective 3 (End-to-end integration):** Persist every run -- its dataset reference, parameters, metrics, model type, and status -- in a database, and surface it in the existing Lumen frontend as a run history and a run-detail view, so that the chain from CSV upload to displayed metrics is demonstrable end-to-end without any hardcoded or mocked results.

These three objectives are Specific (scoped to two named modules), Measurable (a dataset either produces a real profile and a run either produces real metrics, or it does not), Achievable within the current sprint using mature libraries (pandas, scikit-learn), Relevant to the platform's core workflow described in the project proposal, and Time-bound to the "Foundations / Core Tracking" phase of the project timeline.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Lumen follows the layered architecture set out in the project proposal: a React/TypeScript single-page frontend communicates over a REST API with a Python backend, which is responsible for persistence, dataset profiling, and model training. For this stage of the project the backend is a focused FastAPI service rather than the full Celery/Redis/MinIO stack described as the long-term target -- that heavier infrastructure is deferred until the AI Copilot and background-job phases of the roadmap actually require asynchronous processing.

The demonstrable workflow implemented so far is: (1) the user uploads a CSV file from the Datasets page; (2) the backend stores the file and profiles it with pandas, writing a structured profile record to the database; (3) the frontend displays that real profile on the dataset detail page; (4) the user opens the Experiments page, selects the profiled dataset, a target column, a model, and hyperparameters, and starts a run; (5) the backend loads the actual CSV, performs a train/test split, trains a real scikit-learn estimator, computes evaluation metrics, and persists the run; (6) the frontend polls for completion and renders the real metrics on the run-detail page, with the run also appearing in run history. Every arrow in this chain carries real data -- there is no point at which a placeholder or hardcoded value is substituted for an actual computation.

2.1.2 Technical Stack

- **Frontend:** React 19, TypeScript, Vite, React Router, TailwindCSS, Recharts -- the existing Lumen prototype UI, extended rather than replaced.
- **Backend:** Python, FastAPI, Uvicorn -- a lightweight REST API layer chosen for direct, in-process access to pandas and scikit-learn without a subprocess boundary.
- **Database:** SQLite for the current prototype stage (via SQLAlchemy models), with the schema designed so that migrating to PostgreSQL -- as targeted in the full project proposal -- requires no structural change.
- **Data and ML libraries:** pandas for CSV parsing and dataset profiling; scikit-learn for train/test splitting, model training (e.g. Logistic Regression, Random Forest), and metric computation.
- **Tools:** Vite dev server and npm for the frontend; pip/uvicorn for the backend; curl-based manual integration testing of the API during development.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core functionality delivered in this sprint spans the two foundational modules of the platform: **Dataset Intelligence** and **Experiment Tracking with Model Training**. They are implemented as a single connected pipeline rather than two isolated features, since every later module in the proposal (root-cause analysis, explainability, the knowledge graph) depends on a dataset and a run already existing in the system.

Dataset Intelligence. When a CSV is uploaded, the backend saves the file, reads it with pandas, and computes a structured profile: total row and column counts, per-column inferred type (numeric, categorical, or datetime-like), a missing-value count and percentage per column, the number of exact duplicate rows, and descriptive statistics (mean, standard deviation, minimum, maximum for numeric columns; unique-value counts for categorical columns). From these raw statistics the service derives a short list of data-quality observations -- for example flagging a column whose missing-value rate exceeds a threshold, or a column that is constant across every row and therefore useless as a model feature. The profile is stored as a dataset record so it can be looked up by id, both for display and for later selection during experiment configuration.

Experiment Tracking and Model Training. Once a dataset has been profiled, it becomes selectable in the experiment-configuration flow. The user picks a target column, a model type, and a small set of hyperparameters appropriate to that model. On submission, the backend loads the actual stored CSV (not a cached preview), performs a train/test split, fits the chosen scikit-learn estimator on the training split, and evaluates it on the held-out test split to obtain accuracy, precision, recall, and F1 score. The resulting run record -- dataset reference, model type, hyperparameters, computed metrics, status, and timestamps -- is persisted, and the same record is what the frontend's run-history and run-detail views read from. Because the metrics are produced by an actual model fit on an actual dataset split, two runs against the same dataset with different hyperparameters produce genuinely different, comparable results, which is the basic guarantee the rest of the platform's intelligence features will build on.